

COMP 4958: Lab 3

Submit a zip file named `lab3.zip` containing the file `arithmetic.ex` and the `card` folder with its files (but not the `_build` directory). Maximum score: 14

1. Implement the “card server” from lab 2 as a registered GenServer. Use a mix project named `card` and put your code in a module named `Card.Worker`. Note that since this server is registered, the functions in its client API will not need to take a PID as an argument:

```
start()          # start a registered server with "sorted" deck of 52 cards
new()            # use a new "sorted" deck of 52 cards
shuffle()        # shuffle deck of (remaining) cards
count() => integer # return number of remaining cards
deal(n \\ 1) => {:ok, [card]} | {:error, reason} # ask server to deal n cards
```

Note that we can run only one copy of this server since it is registered.

2. Execution within a single process is sequential. A server process that performs lengthy operations may not be able to handle a large volume of requests within a short period. One solution is to have a server that creates a pool of worker processes with the server dispatching requests to the workers. This works particularly well if the workers do not need to store state.

There are different ways of dispatching requests to the workers. One way is round-robin: The first request goes to worker 1, the next to worker 2, and so on, until we get to the last worker. Then we wrap around back to worker 1.

For this exercise, start with the GenServer `Arithmetic.Worker` module from class. Modify the `square` and `sqrt` functions so that the worker returns its PID together with the result of the calculation. (This is to make it easy to see which worker is handling a particular request.) To simulate a lengthy calculation, have the worker sleep for 2 seconds when handling `sqrt` requests.

In a separate module named `Arithmetic.Server`, implement the registered GenServer. Its `start` function takes a positive integer that specifies the number of workers to start. The PIDs of the workers are displayed when they start. This server provides the same operations as the workers: `square(x)` and `sqrt(x)`. However, these only take the number as an argument. (A PID argument is not required as the server is registered.) The server chooses a worker (in round-robin fashion) to handle the (square or square root) request. The server must not be blocked by the chosen worker’s calculation. (Only the chosen worker is blocked when calculating the square root.) The role of the server is to choose a worker; it does not call (or create new processes to call) the `square` or `sqrt` function of the chosen worker.

Put both `Arithmetic.Worker` and `Arithmetic.Server` in a file named `arithmetic.ex`.

The following shows how we can test this:

```
iex(1)> Arithmetic.Server.start(3)
#PID<0.119.0>
#PID<0.120.0>
#PID<0.121.0>
{:ok, #PID<0.118.0>}}
iex(2)> Enum.each(1..3, fn x -> spawn(fn -> IO.inspect(Arithmetic.Server.sqrt(x)) end)
end)
:ok
{#PID<0.121.0>, {:ok, 1.7320508075688772}}
{#PID<0.120.0>, {:ok, 1.4142135623730951}}
{#PID<0.119.0>, {:ok, 1.0}}
iex(3)>
```

Note that we create separate processes to request the 3 square roots. All 3 square roots should appear at about the same time after (approximately) 2 seconds.